

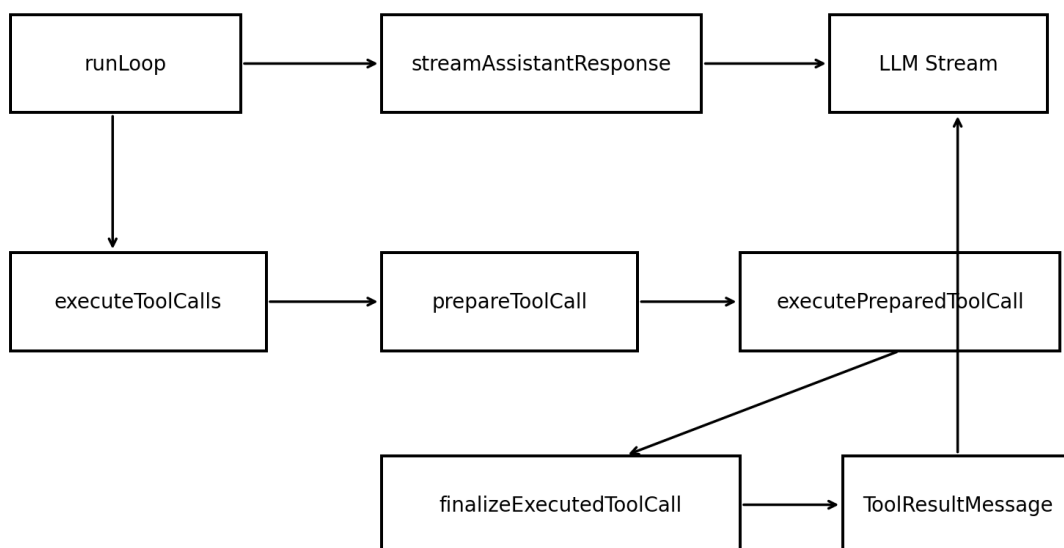
# Pi `agent-loop.ts` 深潜

逐函数逐句源码分析 + 教学版 Python Agent Runtime + s01/s02/s03 一行一行映射

工程学习手册 · 2026-08-13

## Pi agent-loop.ts: six key functions and feedback path

Model call and tool execution are separate pipelines joined by ToolResultMessage



**核心判断：**目标不是“读懂 Pi”，而是把 Pi 的执行语义压缩成可自己维护、可测试、可扩展的 Runtime 骨架。

源码基线：earendil-works/pi / packages/agent/src/agent-loop.ts；分析基于 2026-08-13 获取的 main 版本，文件 blob SHA: a251fede0a9adb9c6cf5e2ba57b4ea2f65b8100b。

版权说明：本文不大段复制原 TypeScript 源码；“逐行”采用执行顺序的语义拆解。完整代码部分是为教学重新实现的 Python 版本，不是官方源码翻译或移植。

# 目录与学习路径

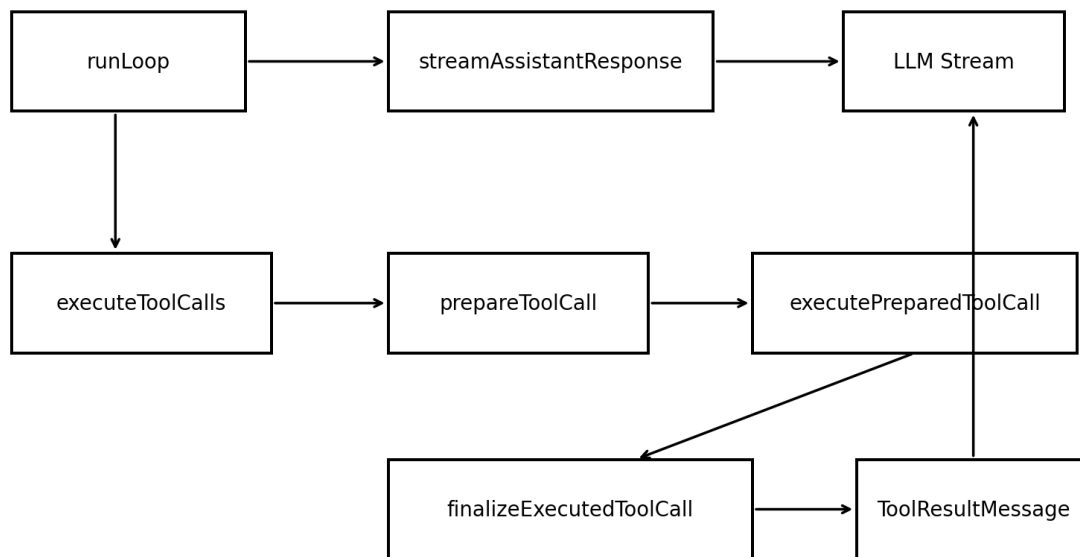
1. 先建立 agent-loop.ts 的正确心智模型
2. s01/s02/s03 到 Pi 的能力增量
3. runLoop：双层循环与生命周期
4. streamAssistantResponse：Context Projection 与流式状态
5. executeToolCalls：调度、并行、顺序与截断安全
6. prepareToolCall：Tool Registry + Validation + Permission
7. executePreparedToolCall：真正的副作用边界
8. finalizeExecutedToolCall：Post Hook 与结果治理
9. 约 300 行核心语义的教学版 Python Runtime
10. s01/s02/s03 一行一行映射
11. Runtime know-how：应该抄什么、不要抄什么
12. 练习与重构路线

# 1. 先建立 agent-loop.ts 的正确心智模型

Pi 的 agent-loop.ts 不负责 Coding Agent 的全部能力。它只承担一个“机械执行内核”：维护单次运行中的上下文、向模型发请求、执行 Tool Call、把 Tool Result 写回、发出生命周期事件，并在必要时继续下一轮。Session、Compaction、Extension、持久化等更高层能力由上层 Agent/AgentSession 接管。

## Pi agent-loop.ts: six key functions and feedback path

Model call and tool execution are separate pipelines joined by ToolResultMessage

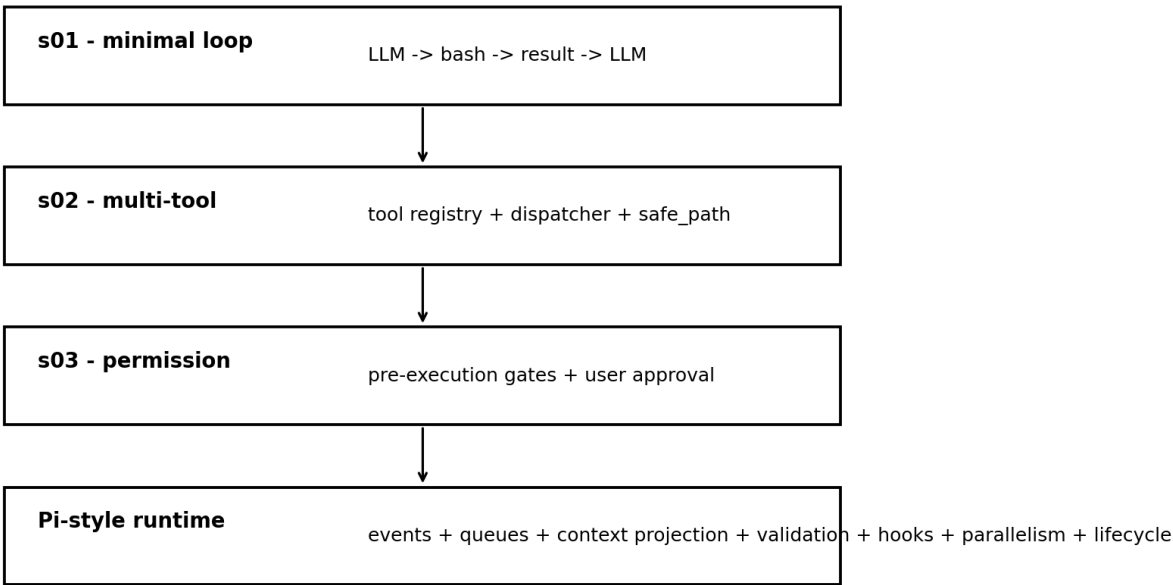


- runLoop 是 orchestrator：决定“还要不要继续下一轮”。
- streamAssistantResponse 是 provider boundary：把 Runtime Context 投影成 LLM Context，并维护 partial/final message。
- executeToolCalls 是 scheduler：选择顺序还是并行执行，并保持结果顺序。
- prepareToolCall 是 policy boundary：工具查找、参数预处理、参数校验、beforeToolCall。
- executePreparedToolCall 是 side-effect boundary：真正调用工具，并把进度更新转成事件。
- finalizeExecutedToolCall 是 result policy boundary：afterToolCall 可重写 content/details/usage/terminate/isError。

**核心判断：Agent Loop 应该尽量“笨”：**复杂性通过可插拔边界进入，而不是持续把业务逻辑塞进 while 循环。

## 2. 从 learn-claude-code s01/s02/s03 到 Pi

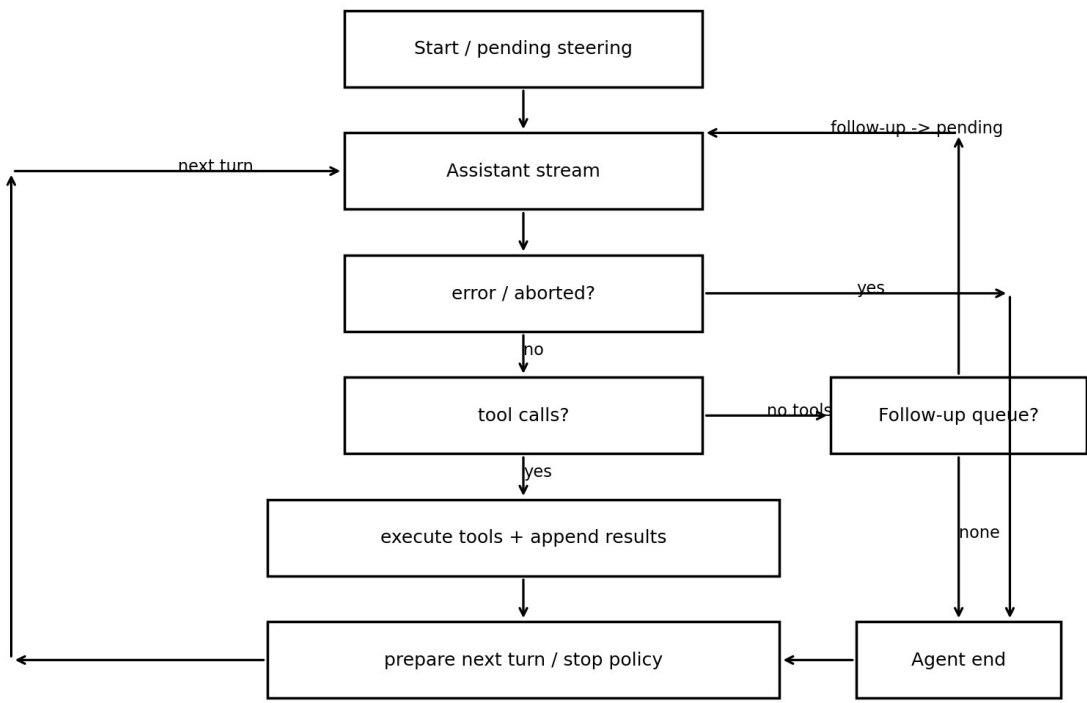
### From learn-claude-code to a reusable Agent Runtime



| 阶段    | 最小新增能力                     | Pi 中的位置                                 | 为什么这是自然演进                                             |
|-------|----------------------------|-----------------------------------------|-------------------------------------------------------|
| s01   | 模型-工具-结果循环                 | runLoop + streamAssistantResponse       | 建立反馈闭环；Tool Result 必须回到模型上下文。                         |
| s02   | 多工具 + 分发                   | executeToolCalls + tool registry lookup | 从硬编码 run_bash 升级为 Tool 抽象和统一 dispatch。                |
| s03   | 执行前权限                      | prepareToolCall -> beforeToolCall       | Permission 是 Tool Pipeline 的 pre-hook，而不是 Bash 的特殊逻辑。 |
| Pi 增量 | 事件/流式/并行/队列/动态刷新/post-hook | 整个 agent-loop.ts                        | 把“能跑”提升到“可观察、可干预、可治理、可嵌入”。                            |

### 3. runLoop：双层循环与生命周期

runLoop = inner ReAct loop + outer follow-up loop



教学版 Python 对应行：157-229。下面按 Pi 源函数的执行顺序逐句拆解，再映射到 Python。

| 源码执行语义                                              | 为什么存在                                                           | 与 s01/s02/s03 的关系                       |
|-----------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------|
| 复制 currentContext / config 为可替换局部变量                 | 允许 prepareNextTurn 在循环中替换 Context / Model / reasoning，而不修改函数签名。 | s01 没有动态配置；这是生产 Runtime 的 late binding。 |
| 启动时先 poll steering                                  | 用户可能在 Agent 等待期间已经输入新指令；不能等一次 LLM 调用后才发现。                       | 超出 s01-s03：Human-in-loop queue。         |
| outer while(true)                                   | 当 Agent 本来准备自然结束时，follow-up 可以重新启动 inner loop。                  | s01 只有一层 while。                         |
| inner while(hasMoreToolCalls    pendingMessages)    | 继续条件不再只有 tool_use，还包括人工 steering。                               | s01 的 stop_reason==tool_use 被扩展。        |
| pending messages 在下次 LLM 前注入                        | 避免中途篡改 provider request；干预发生在明确的 turn boundary。                 | 可测试、可追踪。                                |
| error/aborted 立即 turn_end + agent_end               | 错误也是正常生命周期的一部分，不应靠异常穿透所有层。                                      | s01 往往直接 crash。                         |
| length stop -> fail truncated tool calls            | 可解析不代表完整；副作用工具必须 fail closed。                                   | s03 的安全思想扩大到模型输出完整性。                    |
| append tool results to currentContext + newMessages | currentContext 是下次推理状态；newMessages 是本次运行增量。                     | 比 s01 的单一 messages 更清晰。                 |
| prepareNextTurn                                     | 每轮重新绑定 tools/system/model/thinking 等动态状态。                       | 防止长运行使用 stale snapshot。                 |
| shouldStopAfterTurn                                 | 提供外部策略强制结束，不必侵入 loop。                                           | 机制与策略分离。                                |
| inner loop 结束后 poll follow-up                       | 区分 steer 与 follow-up 的时序语义。                                     | Human-in-loop 的两个队列。                    |

## 对应教学版 Python

```
0157 async def run_loop(
0158     context: AgentContext, new_messages: list[Message], config: Config,
0159     signal: AbortSignal, emit: EventSink, stream_fn: StreamFn,
0160 ) -> None:
0161     first_turn = True
0162     pending = await config.get_steering_messages() if config.get_steering_messages else []
0163     while True:
0164         has_more_tool_calls = True
0165         while has_more_tool_calls or pending:
0166             # inner: ReAct + steering
0167             if first_turn:
0168                 first_turn = False
0169             else:
0170                 await emit(Event("turn_start"))
0171                 for msg in pending:
0172                     # inject steering
0173                     await emit(Event("message_start", {"message": msg}))
0174                     await emit(Event("message_end", {"message": msg}))
0175                     context.messages.append(msg)
0176                     new_messages.append(msg)
0177                 pending = []
0178             assistant = await stream_assistant_response(
0179                 context, config, signal, emit, stream_fn
0180             )
0181             new_messages.append(assistant)
0182             if assistant.stop_reason in {"error", "aborted"}:
0183                 await emit(Event("turn_end", {"message": assistant, "tool_results": []}))
0184                 await emit(Event("agent_end", {"messages": new_messages}))
0185                 return
0186             calls = assistant.tool_calls()
0187             result_messages: list[ToolResultMessage] = []
0188             has_more_tool_calls = False
0189             if calls:
0190                 batch = (
0191                     await fail_truncated_calls(calls, emit)
0192                     if assistant.stop_reason == "length"
0193                     else await execute_tool_calls(context, assistant, config, signal, emit)
0194                 )
0195                 result_messages.extend(batch.messages)
0196                 has_more_tool_calls = not batch.terminate
0197                 context.messages.extend(result_messages)
0198                 new_messages.extend(result_messages)
0199             await emit(Event("turn_end", {
0200                 "message": assistant, "tool_results": result_messages,
0201             }))
0202             turn = {
0203                 "message": assistant, "tool_results": result_messages,
0204                 "context": context, "new_messages": new_messages,
0205             }
0206             if config.prepare_next_turn:
0207                 # late-bind runtime state
0208                 snap = await config.prepare_next_turn(turn)
0209                 if snap:
0210                     context = snap.context or context
0211                     if snap.model is not None:
0212                         config = replace(config, model=snap.model)
0213                     if snap.thinking_level is not None:
0214                         level = None if snap.thinking_level == "off" else snap.thinking_level
0215                         config = replace(config, reasoning=level)
0216             if config.should_stop_after_turn and await config.should_stop_after_turn(turn):
0217                 await emit(Event("agent_end", {"messages": new_messages}))
0218                 return
0219             pending = (
0220                 await config.get_steering_messages()
0221                 if config.get_steering_messages else []
0222             )
0223             follow_ups = (
0224                 await config.get_follow_up_messages()
0225                 if config.get_follow_up_messages else []
0226             )
0227             if follow_ups:
0228                 pending = follow_ups
0229                 continue
0230             break
0231         await emit(Event("agent_end", {"messages": new_messages}))
```

# 4. streamAssistantResponse：Context Projection 与流式状态

教学版 Python 对应行：230-287。下面按 Pi 源函数的执行顺序逐句拆解，再映射到 Python。

| 源码执行语义                                | 为什么存在                                                                 | 与 s01/s02/s03 的关系                       |
|---------------------------------------|-----------------------------------------------------------------------|-----------------------------------------|
| transformContext                      | Runtime message -> Runtime message；适合裁剪、RAG、Memory、Extension context。 | Context Engineering 层。                  |
| convertToLlm                          | Runtime message -> provider-compatible message；是协议投影，不是上下文策略。         | 将 Runtime State 与 Provider Protocol 分开。 |
| 构造 systemPrompt/messages/tools        | LLM Context 是 Runtime Context 的投影，不是 Session 的镜像。                     | 关键 know-how。                            |
| 每次请求动态 resolve API key                | OAuth/短期 token 会过期；启动时缓存 key 不够。                                      | Provider lifecycle。                     |
| start 时把 partial message 直接放入 context | Agent state 在 streaming 时也可观察，而不是 UI 自己维护 shadow state。               | Event-driven UI。                        |
| delta 时替换 context 最后一条                | 保留“当前位置就是最新 partial”这一不变量。                                            | 减少双状态同步。                                |
| done/error 统一 result()                | provider stream 的终态都通过 final message 收敛。                              | 统一错误/成功模型。                              |
| 无 terminal event 的 fallback           | 第三方 provider adapter 不完美时仍能收敛 final state。                            | defensive runtime。                      |

## 对应教学版 Python

```
0230 async def stream_assistant_response(  
0231     context: AgentContext, config: Config, signal: AbortSignal,  
0232     emit: EventSink, stream_fn: StreamFn,  
0233 ) -> Message:  
0234     messages = context.messages  
0235     if config.transform_context:  
0236         messages = await config.transform_context(messages, signal)  
0237     converter = config.convert_to_llm or default_convert  
0238     llm_messages = await converter(messages)  
0239     llm_context = {  
0240         "system_prompt": context.system_prompt,  
0241         "messages": llm_messages,  
0242         "tools": context.tools,  
0243     }  
0244     api_key = config.api_key  
0245     provider = getattr(config.model, "provider", "unknown")  
0246     if config.get_api_key:  
0247         api_key = await config.get_api_key(provider) or api_key  
0248     stream = await stream_fn(config.model, llm_context, {  
0249         "api_key": api_key, "reasoning": config.reasoning, "signal": signal,  
0250     })  
0251     partial: Message | None = None  
0252     added_partial = False  
0253     async for ev in stream:  
0254         if ev.type == "start":  
0255             partial = ev.partial  
0256             context.messages.append(partial)  
0257             added_partial = True  
0258             await emit(Event("message_start", {"message": partial}))  
0259             continue  
0260         if ev.type in {  
0261             "text_start", "text_delta", "text_end",  
0262             "thinking_start", "thinking_delta", "thinking_end",  
0263             "toolcall_start", "toolcall_delta", "toolcall_end",  
0264         }:  
0265             if partial is not None:  
0266                 partial = ev.partial  
0267                 context.messages[-1] = partial  
0268                 await emit(Event("message_update", {"message": partial, "raw": ev}))  
0269             continue  
0270         if ev.type in {"done", "error"}:  
0271             final = await stream.result()  
0272             if added_partial:
```

```

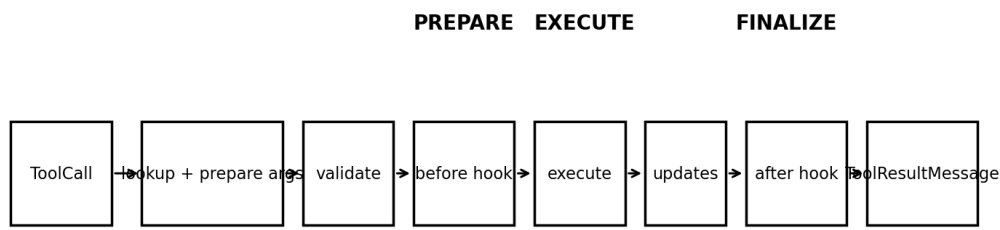
0273         context.messages[-1] = final
0274     else:
0275         context.messages.append(final)
0276         await emit(Event("message_start", {"message": final}))
0277         await emit(Event("message_end", {"message": final}))
0278         return final
0279     final = await stream.result()           # defensive fallback
0280     if added_partial:
0281         context.messages[-1] = final
0282     else:
0283         context.messages.append(final)
0284         await emit(Event("message_start", {"message": final}))
0285         await emit(Event("message_end", {"message": final}))
0286         return final
0287

```



# 5. executeToolCalls：调度、并行、顺序与截断安全

Tool execution is a pipeline, not a single function call



s02 adds lookup/dispatch; s03 naturally lands at before hook; production runtime adds validation, updates, post-processing and scheduling.

教学版 Python 对应行：301-378。下面按 Pi 源函数的执行顺序逐句拆解，再映射到 Python。

| 源码执行语义                                                       | 为什么存在                                   | 与 s01/s02/s03 的关系                |
|--------------------------------------------------------------|-----------------------------------------|----------------------------------|
| 从 assistant.content 提取 toolCall                              | 只执行结构化调用；普通文本不参与 Tool Runtime。          | s01 的 stop_reason 进一步结构化。        |
| 扫描 tool.executionMode                                        | 任何一个工具要求 sequential，则整批按顺序执行。           | 语义元数据影响调度。                       |
| 全局 config.toolExecution 也可强制 sequential                      | Host 可以在测试/安全模式关闭并行。                    | Runtime policy。                  |
| sequential: prepare -> execute -> finalize -> result message | 严格保持顺序和每一步事件。                           | 与 s02 dispatcher 同源，但更完整。        |
| parallel: 先顺序 prepare，再并行真正 execute                          | 权限/校验有确定顺序；副作用阶段才并发。                    | 并行不等于所有阶段 Promise.all。           |
| Promise.all/gather 保持输入顺序                                    | 物理完成顺序可不同，但 ToolResultMessage 仍按调用顺序写回。 | 降低 provider/message ordering 风险。 |
| terminate 是 batch-level reduction                            | 只有整批工具都声明 terminate 才终止继续循环。            | 避免一个工具意外吞掉其他结果。                  |

## 对应教学版 Python

```
0301 async def execute_tool_calls(  
0302     context: AgentContext, assistant: Message, config: Config,  
0303     signal: AbortSignal, emit: EventSink,  
0304 ) -> Batch:  
0305     calls = assistant.tool_calls()  
0306     registry = {tool.name: tool for tool in context.tools}  
0307     requires_order = any(  
0308         registry.get(c.name) and registry[c.name].execution_mode == "sequential"  
0309         for c in calls  
0310     )  
0311     if config.tool_execution == "sequential" or requires_order:  
0312         return await execute_sequential(context, assistant, calls, config, signal, emit)  
0313     return await execute_parallel(context, assistant, calls, config, signal, emit)  
0314  
0315 async def execute_sequential(  
0316     context: AgentContext, assistant: Message, calls: list[ToolCall],  
0317     config: Config, signal: AbortSignal, emit: EventSink,  
0318 ) -> Batch:  
0319     finalized: list[Finalized] = []  
0320     messages: list[ToolResultMessage] = []  
0321     for call in calls:
```

```

0322     await emit(Event("tool_execution_start", {"call": call}))
0323     prepared = await prepare_tool_call(context, assistant, call, config, signal)
0324     if isinstance(prepared, Immediate):
0325         item = Finalized(call, prepared.result, prepared.is_error)
0326     else:
0327         executed = await execute_prepared_tool_call(prepared, signal, emit)
0328         item = await finalize_executed_tool_call(
0329             context, assistant, prepared, executed, config, signal
0330         )
0331     await emit(Event("tool_execution_end", {"outcome": item}))
0332     msg = as_result_message(item)
0333     await emit_result_message(msg, emit)
0334     finalized.append(item)
0335     messages.append(msg)
0336     if signal.aborted:
0337         break
0338     return Batch(messages, terminate_batch(finalized))
0339
0340 async def execute_parallel(
0341     context: AgentContext, assistant: Message, calls: list[ToolCall],
0342     config: Config, signal: AbortSignal, emit: EventSink,
0343 ) -> Batch:
0344     entries: list[Finalized | Prepared] = []
0345     for call in calls:
0346         # prepare/policy remains ordered
0347         await emit(Event("tool_execution_start", {"call": call}))
0348         prepared = await prepare_tool_call(context, assistant, call, config, signal)
0349         if isinstance(prepared, Immediate):
0350             item = Finalized(call, prepared.result, prepared.is_error)
0351             await emit(Event("tool_execution_end", {"outcome": item}))
0352             entries.append(item)
0353         else:
0354             entries.append(prepared)
0355             if signal.aborted:
0356                 break
0357     async def run_one(p: Prepared) -> Finalized:
0358         executed = await execute_prepared_tool_call(p, signal, emit)
0359         item = await finalize_executed_tool_call(
0360             context, assistant, p, executed, config, signal
0361         )
0362         await emit(Event("tool_execution_end", {"outcome": item}))
0363         return item
0364     jobs: list[Awaitable[Finalized]] = []
0365     for entry in entries:
0366         if isinstance(entry, Finalized):
0367             async def ready(value: Finalized = entry) -> Finalized:
0368                 return value
0369             jobs.append(ready())
0370         else:
0371             jobs.append(run_one(entry))
0372     finalized = await asyncio.gather(*jobs) # preserves call order
0373     messages: list[ToolResultMessage] = []
0374     for item in finalized:
0375         msg = as_result_message(item)
0376         await emit_result_message(msg, emit)
0377         messages.append(msg)
0378     return Batch(messages, terminate_batch(finalized))

```

## 6. prepareToolCall : Tool Registry + Validation + Permission

教学版 Python 对应行：379-404。下面按 Pi 源函数的执行顺序逐句拆解，再映射到 Python。

| 源码执行语义                              | 为什么存在                                       | 与 s01/s02/s03 的关系            |
|-------------------------------------|---------------------------------------------|------------------------------|
| 按 name 查 Tool Registry              | s02 的 TOOL_HANDLERS 映射升级为 Tool 对象 registry。 | 多工具基础。                       |
| 找不到工具 -> error ToolResult, 不抛到 loop | 让模型看到可恢复 observation。                       | Agent 错误应尽量回到 feedback loop。 |
| prepareArguments                    | 允许兼容旧参数、默认值、轻量 normalization。               | Schema 前处理。                  |
| validateToolArguments               | 参数在副作用发生前验证。                                | s02 常只靠 handler 自己报错。        |
| beforeToolCall                      | 权限、审批、路径策略、配额等统一插入点。                        | s03 check_permission 的标准落点。  |
| block -> ImmediateOutcome           | 被拦截与真正执行共享同一 ToolResult 生命周期。               | UI/LLM 不需要特殊分支。              |
| abort 检查放在 hook 前后                  | 用户可在等待审批/策略判断时取消。                           | 协作式取消。                       |
| try/catch -> ToolResult error       | 校验/策略错误不会炸掉 Agent Runtime。                  | resilience。                  |

### 对应教学版 Python

```
0379 async def prepare_tool_call(
0380     context: AgentContext, assistant: Message, call: ToolCall,
0381     config: Config, signal: AbortSignal,
0382 ) -> Prepared | Immediate:
0383     tool = next((x for x in context.tools if x.name == call.name), None)
0384     if tool is None:
0385         return Immediate(error_result(f"tool not found: {call.name}"), True)
0386     try:
0387         args = tool.prepare_arguments(call.arguments)
0388         args = tool.validate_arguments(args)
0389         if config.before_tool_call:
0390             decision = await config.before_tool_call(
0391                 assistant, call, args, context, signal
0392             )
0393             if signal.aborted:
0394                 return Immediate(error_result("operation aborted"), True)
0395             if decision and decision.block:
0396                 result = error_result(decision.reason or "tool execution blocked")
0397                 result.terminate = decision.terminate
0398                 return Immediate(result, True)
0399             if signal.aborted:
0400                 return Immediate(error_result("operation aborted"), True)
0401             return Prepared(call, tool, args)
0402     except Exception as exc:
0403         return Immediate(error_result(str(exc)), True)
0404
```

## 7. executePreparedToolCall：真正的副作用边界

教学版 Python 对应行：405-434。下面按 Pi 源函数的执行顺序逐句拆解，再映射到 Python。

| 源码执行语义                                | 为什么存在                                   | 与 s01/s02/s03 的关系 |
|---------------------------------------|-----------------------------------------|-------------------|
| 只有 Prepared 才能进入 execute              | 副作用边界之前已完成 lookup/validation/policy。    | 强不变量。             |
| on_update -> tool_execution_update    | 长工具可以流式反馈进度。                            | 避免“黑盒等待”。         |
| acceptingUpdates 开关                   | 工具结束后迟到 callback 不应继续污染事件流。             | 并发边界治理。           |
| 等待所有 update event settle              | 保证 tool_execution_end 不会跑到早先 update 前面。 | 事件顺序不变量。          |
| execute exception -> error ToolResult | Tool 失败是 observation，不是 Runtime 崩溃。     | s01/s02 常见提升点。    |

### 对应教学版 Python

```
0405 async def execute_prepared_tool_call(  
0406     prepared: Prepared, signal: AbortSignal, emit: EventSink,  
0407 ) -> Executed:  
0408     update_jobs: list[asyncio.Task[None]] = []  
0409     accepting_updates = True  
0410     def on_update(partial: ToolResult) -> None:  
0411         if not accepting_updates:  
0412             return  
0413         update_jobs.append(asyncio.create_task(emit(Event("tool_execution_update", {  
0414             "tool_call_id": prepared.call_id,  
0415             "tool_name": prepared.call.name,  
0416             "args": prepared.call.arguments,  
0417             "partial_result": partial,  
0418         }))))  
0419     try:  
0420         result = await prepared.tool.execute(  
0421             prepared.call.id, prepared.args, signal, on_update  
0422         )  
0423         accepting_updates = False  
0424         if update_jobs:  
0425             await asyncio.gather(*update_jobs)  
0426         return Executed(result, False)  
0427     except Exception as exc:  
0428         accepting_updates = False  
0429         if update_jobs:  
0430             await asyncio.gather(*update_jobs, return_exceptions=True)  
0431         return Executed(error_result(str(exc)), True)  
0432     finally:  
0433         accepting_updates = False  
0434
```

## 8. finalizeExecutedToolCall：Post Hook 与结果治理

教学版 Python 对应行：435-460。下面按 Pi 源函数的执行顺序逐句拆解，再映射到 Python。

| 源码执行语义                               | 为什么存在                                                | 与 s01/s02/s03 的关系                 |
|--------------------------------------|------------------------------------------------------|-----------------------------------|
| 复制 result / isError 为可重写局部值          | post-hook 可做治理但不必修改 Tool 实现。                         | 机制/策略分离。                          |
| afterToolCall 接收 args/result/context | 支持审计、redaction、truncation、usage、artifact extraction。 | Tool Result 是第二个 policy boundary。 |
| 字段级 fallback                         | hook 只改自己关心的字段，其他沿用原结果。                              | Composable extension。             |
| after hook 自身失败 -> error ToolResult  | Extension 失败也不能破坏主循环。                                | failure containment。              |
| 返回 FinalizedOutcome                  | 把执行结果标准化后再生成 ToolResultMessage。                      | 后续持久化/LLM/UI 共享稳定结构。              |

### 对应教学版 Python

```
0435 async def finalize_executed_tool_call(  
0436     context: AgentContext, assistant: Message, prepared: Prepared,  
0437     executed: Executed, config: Config, signal: AbortSignal,  
0438 ) -> Finalized:  
0439     result, is_error = executed.result, executed.is_error  
0440     if config.after_tool_call:  
0441         try:  
0442             decision = await config.after_tool_call(  
0443                 assistant, prepared.call, prepared.args,  
0444                 result, is_error, context, signal,  
0445             )  
0446             if decision:  
0447                 result = ToolResult(  
0448                     content=decision.content if decision.content is not None else result.content,  
0449                     details=decision.details if decision.details is not None else result.details,  
0450                     usage=decision.usage if decision.usage is not None else result.usage,  
0451                     terminate=(  
0452                         decision.terminate if decision.terminate is not None  
0453                         else result.terminate  
0454                     ),  
0455                 )  
0456             if decision.is_error is not None:  
0457                 is_error = decision.is_error  
0458         except Exception as exc:  
0459             result, is_error = error_result(str(exc)), True  
0460     return Finalized(prepared.call, result, is_error)
```

## 9. 完整教学版 Python Agent Runtime

文件共 460 个物理行；其中类型声明、空行和为了 PDF 可读性的换行占了一部分，核心控制流约 300 行。代码通过 `python -m py_compile` 语法检查。

**核心判断：这份代码不是要“替代 Pi”，而是把 Pi 的六个关键函数压缩成一个可单步调试、可继续演进的 Python 骨架。**

```
0001 """Teaching-only Python runtime that preserves Pi agent-loop semantics."""
0002 from __future__ import annotations
0003 import asyncio
0004 from dataclasses import dataclass, field, replace
0005 from typing import Any, AsyncIterator, Awaitable, Callable, Literal, Protocol
0006
0007 ExecutionMode = Literal["parallel", "sequential"]
0008
0009 @dataclass
0010 class ToolCall:
0011     id: str
0012     name: str
0013     arguments: dict[str, Any]
0014
0015 @dataclass
0016 class Message:
0017     role: str
0018     content: list[Any]
0019     stop_reason: str | None = None
0020     def tool_calls(self) -> list[ToolCall]:
0021         return [x for x in self.content if isinstance(x, ToolCall)]
0022
0023 @dataclass
0024 class ToolResult:
0025     content: list[Any]
0026     details: dict[str, Any] = field(default_factory=dict)
0027     usage: dict[str, Any] | None = None
0028     terminate: bool = False
0029
0030 @dataclass
0031 class ToolResultMessage(Message):
0032     tool_call_id: str = ""
0033     tool_name: str = ""
0034     is_error: bool = False
0035
0036 @dataclass
0037 class Event:
0038     type: str
0039     data: dict[str, Any] = field(default_factory=dict)
0040
0041 class AbortSignal:
0042     def __init__(self) -> None:
0043         self.aborted = False
0044     def abort(self) -> None:
0045         self.aborted = True
0046
0047 class Tool(Protocol):
0048     name: str
0049     execution_mode: ExecutionMode
0050     def prepare_arguments(self, raw: dict[str, Any]) -> dict[str, Any]: ...
0051     def validate_arguments(self, args: dict[str, Any]) -> dict[str, Any]: ...
0052     async def execute(
0053         self, call_id: str, args: dict[str, Any], signal: AbortSignal,
0054         on_update: Callable[[ToolResult], None],
0055     ) -> ToolResult: ...
0056
0057 @dataclass
0058 class StreamEvent:
0059     type: str
0060     partial: Message
0061
0062 class ModelStream(Protocol):
0063     def __aiter__(self) -> AsyncIterator[StreamEvent]: ...
0064     async def result(self) -> Message: ...
0065
0066 EventSink = Callable[[Event], Awaitable[None]]
0067 StreamFn = Callable[[Any, dict[str, Any], dict[str, Any]], Awaitable[ModelStream]]
0068
0069 @dataclass
0070 class AgentContext:
0071     system_prompt: str
0072     messages: list[Message]
```

```

0073     tools: list[Tool]
0074
0075 @dataclass
0076 class BeforeDecision:
0077     block: bool = False
0078     reason: str | None = None
0079     terminate: bool = False
0080
0081 @dataclass
0082 class AfterDecision:
0083     content: list[Any] | None = None
0084     details: dict[str, Any] | None = None
0085     usage: dict[str, Any] | None = None
0086     terminate: bool | None = None
0087     is_error: bool | None = None
0088
0089 @dataclass
0090 class NextTurn:
0091     context: AgentContext | None = None
0092     model: Any | None = None
0093     thinking_level: str | None = None
0094
0095 @dataclass
0096 class Config:
0097     model: Any
0098     api_key: str | None = None
0099     reasoning: str | None = None
0100     tool_execution: ExecutionMode = "parallel"
0101     transform_context: Callable[..., Awaitable[list[Message]]] | None = None
0102     convert_to_llm: Callable[[list[Message]], Awaitable[list[Message]]] | None = None
0103     get_api_key: Callable[[str], Awaitable[str | None]] | None = None
0104     before_tool_call: Callable[..., Awaitable[BeforeDecision | None]] | None = None
0105     after_tool_call: Callable[..., Awaitable[AfterDecision | None]] | None = None
0106     prepare_next_turn: Callable[..., Awaitable[NextTurn | None]] | None = None
0107     should_stop_after_turn: Callable[..., Awaitable[bool]] | None = None
0108     get_steering_messages: Callable[[], Awaitable[list[Message]]] | None = None
0109     get_follow_up_messages: Callable[[], Awaitable[list[Message]]] | None = None
0110
0111 @dataclass
0112 class Prepared:
0113     call: ToolCall
0114     tool: Tool
0115     args: dict[str, Any]
0116
0117 @dataclass
0118 class Immediate:
0119     result: ToolResult
0120     is_error: bool
0121
0122 @dataclass
0123 class Executed:
0124     result: ToolResult
0125     is_error: bool
0126
0127 @dataclass
0128 class Finalized:
0129     call: ToolCall
0130     result: ToolResult
0131     is_error: bool
0132
0133 @dataclass
0134 class Batch:
0135     messages: list[ToolResultMessage]
0136     terminate: bool
0137
0138 def error_result(text: str) -> ToolResult:
0139     return ToolResult([{"type": "text", "text": text}])
0140
0141 def as_result_message(x: Finalized) -> ToolResultMessage:
0142     return ToolResultMessage(
0143         role="tool_result", content=x.result.content or [],
0144         tool_call_id=x.call.id, tool_name=x.call.name, is_error=x.is_error,

```

```

0145     )
0146
0147 def terminate_batch(items: list[Finalized]) -> bool:
0148     return bool(items) and all(x.result.terminate for x in items)
0149
0150 async def default_convert(messages: list[Message]) -> list[Message]:
0151     return [m for m in messages if m.role in {"user", "assistant", "tool_result"}]
0152
0153 async def emit_result_message(msg: ToolResultMessage, emit: EventSink) -> None:
0154     await emit(Event("message_start", {"message": msg}))
0155     await emit(Event("message_end", {"message": msg}))
0156
0157 async def run_loop(
0158     context: AgentContext, new_messages: list[Message], config: Config,
0159     signal: AbortSignal, emit: EventSink, stream_fn: StreamFn,

```

```

0160 ) -> None:
0161     first_turn = True
0162     pending = await config.get_steering_messages() if config.get_steering_messages else []
0163     while True:
0164         has_more_tool_calls = True
0165         while has_more_tool_calls or pending:
0166             if first_turn:
0167                 first_turn = False
0168             else:
0169                 await emit(Event("turn_start"))
0170                 for msg in pending:
0171                     # inject steering
0172                     await emit(Event("message_start", {"message": msg}))
0173                     await emit(Event("message_end", {"message": msg}))
0174                     context.messages.append(msg)
0175                     new_messages.append(msg)
0176                 pending = []
0177                 assistant = await stream_assistant_response(
0178                     context, config, signal, emit, stream_fn
0179                 )
0180                 new_messages.append(assistant)
0181                 if assistant.stop_reason in {"error", "aborted"}:
0182                     await emit(Event("turn_end", {"message": assistant, "tool_results": []}))
0183                     await emit(Event("agent_end", {"messages": new_messages}))
0184                     return
0185                 calls = assistant.tool_calls()
0186                 result_messages: list[ToolResultMessage] = []
0187                 has_more_tool_calls = False
0188                 if calls:
0189                     batch = (
0190                         await fail_truncated_calls(calls, emit)
0191                         if assistant.stop_reason == "length"
0192                         else await execute_tool_calls(context, assistant, config, signal, emit)
0193                     )
0194                     result_messages.extend(batch.messages)
0195                     has_more_tool_calls = not batch.terminate
0196                     context.messages.extend(result_messages)
0197                     new_messages.extend(result_messages)
0198                 await emit(Event("turn_end", {
0199                     "message": assistant, "tool_results": result_messages,
0200                 }))
0201                 turn = {
0202                     "message": assistant, "tool_results": result_messages,
0203                     "context": context, "new_messages": new_messages,
0204                 }
0205                 if config.prepare_next_turn:
0206                     # late-bind runtime state
0207                     snap = await config.prepare_next_turn(turn)
0208                     if snap:
0209                         context = snap.context or context
0210                         if snap.model is not None:
0211                             config = replace(config, model=snap.model)
0212                         if snap.thinking_level is not None:
0213                             level = None if snap.thinking_level == "off" else snap.thinking_level
0214                             config = replace(config, reasoning=level)
0215                 if config.should_stop_after_turn and await config.should_stop_after_turn(turn):
0216                     await emit(Event("agent_end", {"messages": new_messages}))
0217                     return
0218                 pending = (

```

```

0217         await config.get_steering_messages()
0218         if config.get_steering_messages else []
0219     )
0220     follow_ups = (
0221         await config.get_follow_up_messages()
0222         if config.get_follow_up_messages else []
0223     )
0224     if follow_ups:
0225         pending = follow_ups
0226         continue
0227     break
0228     await emit(Event("agent_end", {"messages": new_messages}))
0229
0230 async def stream_assistant_response(
0231     context: AgentContext, config: Config, signal: AbortSignal,
0232     emit: EventSink, stream_fn: StreamFn,
0233 ) -> Message:
0234     messages = context.messages
0235     if config.transform_context:
0236         messages = await config.transform_context(messages, signal)
0237     converter = config.convert_to_llm or default_convert
0238     llm_messages = await converter(messages)
0239     llm_context = {
0240         "system_prompt": context.system_prompt,
0241         "messages": llm_messages,
0242         "tools": context.tools,
0243     }
0244     api_key = config.api_key
0245     provider = getattr(config.model, "provider", "unknown")
0246     if config.get_api_key:

```



```

0247     api_key = await config.get_api_key(provider) or api_key
0248     stream = await stream_fn(config.model, llm_context, {
0249         "api_key": api_key, "reasoning": config.reasoning, "signal": signal,
0250     })
0251     partial: Message | None = None
0252     added_partial = False
0253     async for ev in stream:
0254         if ev.type == "start":
0255             partial = ev.partial
0256             context.messages.append(partial)
0257             added_partial = True
0258             await emit(Event("message_start", {"message": partial}))
0259             continue
0260         if ev.type in {
0261             "text_start", "text_delta", "text_end",
0262             "thinking_start", "thinking_delta", "thinking_end",
0263             "toolcall_start", "toolcall_delta", "toolcall_end",
0264         }:
0265             if partial is not None:
0266                 partial = ev.partial
0267                 context.messages[-1] = partial
0268                 await emit(Event("message_update", {"message": partial, "raw": ev}))
0269                 continue
0270         if ev.type in {"done", "error"}:
0271             final = await stream.result()
0272             if added_partial:
0273                 context.messages[-1] = final
0274             else:
0275                 context.messages.append(final)
0276                 await emit(Event("message_start", {"message": final}))
0277                 await emit(Event("message_end", {"message": final}))
0278             return final
0279     final = await stream.result() # defensive fallback
0280     if added_partial:
0281         context.messages[-1] = final
0282     else:
0283         context.messages.append(final)
0284         await emit(Event("message_start", {"message": final}))
0285         await emit(Event("message_end", {"message": final}))
0286     return final
0287
0288 async def fail_truncated_calls(calls: list[ToolCall], emit: EventSink) -> Batch:

```

```

0289     messages: list[ToolResultMessage] = []
0290     for call in calls:
0291         await emit(Event("tool_execution_start", {"call": call}))
0292         item = Finalized(
0293             call, error_result("not executed: output truncated; retry complete args"), True
0294         )
0295         await emit(Event("tool_execution_end", {"outcome": item}))
0296         msg = as_result_message(item)
0297         await emit_result_message(msg, emit)
0298         messages.append(msg)
0299     return Batch(messages, terminate=False)
0300
0301 async def execute_tool_calls(
0302     context: AgentContext, assistant: Message, config: Config,
0303     signal: AbortSignal, emit: EventSink,
0304 ) -> Batch:
0305     calls = assistant.tool_calls()
0306     registry = {tool.name: tool for tool in context.tools}
0307     requires_order = any(
0308         registry.get(c.name) and registry[c.name].execution_mode == "sequential"
0309         for c in calls
0310     )
0311     if config.tool_execution == "sequential" or requires_order:
0312         return await execute_sequential(context, assistant, calls, config, signal, emit)
0313     return await execute_parallel(context, assistant, calls, config, signal, emit)
0314
0315 async def execute_sequential(
0316     context: AgentContext, assistant: Message, calls: list[ToolCall],
0317     config: Config, signal: AbortSignal, emit: EventSink,
0318 ) -> Batch:
0319     finalized: list[Finalized] = []
0320     messages: list[ToolResultMessage] = []
0321     for call in calls:
0322         await emit(Event("tool_execution_start", {"call": call}))
0323         prepared = await prepare_tool_call(context, assistant, call, config, signal)
0324         if isinstance(prepared, Immediate):
0325             item = Finalized(call, prepared.result, prepared.is_error)
0326         else:
0327             executed = await execute_prepared_tool_call(prepared, signal, emit)
0328             item = await finalize_executed_tool_call(
0329                 context, assistant, prepared, executed, config, signal
0330             )
0331         await emit(Event("tool_execution_end", {"outcome": item}))
0332         msg = as_result_message(item)
0333         await emit_result_message(msg, emit)

```

```

0334         finalized.append(item)
0335         messages.append(msg)
0336         if signal.aborted:
0337             break
0338         return Batch(messages, terminate_batch(finalized))
0339
0340     async def execute_parallel(
0341         context: AgentContext, assistant: Message, calls: list[ToolCall],
0342         config: Config, signal: AbortSignal, emit: EventSink,
0343     ) -> Batch:
0344         entries: list[Finalized | Prepared] = []
0345         for call in calls:
0346             # prepare/policy remains ordered
0347             await emit(Event("tool_execution_start", {"call": call}))
0348             prepared = await prepare_tool_call(context, assistant, call, config, signal)
0349             if isinstance(prepared, Immediate):
0350                 item = Finalized(call, prepared.result, prepared.is_error)
0351                 await emit(Event("tool_execution_end", {"outcome": item}))
0352                 entries.append(item)
0353             else:
0354                 entries.append(prepared)
0355             if signal.aborted:
0356                 break
0357         async def run_one(p: Prepared) -> Finalized:
0358             executed = await execute_prepared_tool_call(p, signal, emit)
0359             item = await finalize_executed_tool_call(
0360                 context, assistant, p, executed, config, signal
0361             )

```

```

0361         await emit(Event("tool_execution_end", {"outcome": item}))
0362         return item
0363     jobs: list[Awaitable[Finalized]] = []
0364     for entry in entries:
0365         if isinstance(entry, Finalized):
0366             async def ready(value: Finalized = entry) -> Finalized:
0367                 return value
0368             jobs.append(ready())
0369         else:
0370             jobs.append(run_one(entry))
0371     finalized = await asyncio.gather(*jobs) # preserves call order
0372     messages: list[ToolResultMessage] = []
0373     for item in finalized:
0374         msg = as_result_message(item)
0375         await emit_result_message(msg, emit)
0376         messages.append(msg)
0377     return Batch(messages, terminate_batch(finalized))
0378
0379     async def prepare_tool_call(
0380         context: AgentContext, assistant: Message, call: ToolCall,
0381         config: Config, signal: AbortSignal,
0382     ) -> Prepared | Immediate:
0383         tool = next((x for x in context.tools if x.name == call.name), None)
0384         if tool is None:
0385             return Immediate(error_result(f"tool not found: {call.name}"), True)
0386         try:
0387             args = tool.prepare_arguments(call.arguments)
0388             args = tool.validate_arguments(args)
0389             if config.before_tool_call:
0390                 decision = await config.before_tool_call(
0391                     assistant, call, args, context, signal
0392                 )
0393                 if signal.aborted:
0394                     return Immediate(error_result("operation aborted"), True)
0395                 if decision and decision.block:
0396                     result = error_result(decision.reason or "tool execution blocked")
0397                     result.terminate = decision.terminate
0398                     return Immediate(result, True)
0399             if signal.aborted:
0400                 return Immediate(error_result("operation aborted"), True)
0401             return Prepared(call, tool, args)
0402         except Exception as exc:
0403             return Immediate(error_result(str(exc)), True)
0404
0405     async def execute_prepared_tool_call(
0406         prepared: Prepared, signal: AbortSignal, emit: EventSink,
0407     ) -> Executed:
0408         update_jobs: list[asyncio.Task[None]] = []
0409         accepting_updates = True
0410         def on_update(partial: ToolResult) -> None:
0411             if not accepting_updates:
0412                 return
0413             update_jobs.append(asyncio.create_task(emit(Event("tool_execution_update", {
0414                 "tool_call_id": prepared.call_id,
0415                 "tool_name": prepared.call.name,
0416                 "args": prepared.call.arguments,
0417                 "partial_result": partial,
0418             }))))
0419         try:
0420             result = await prepared.tool.execute(

```

```

0421         prepared.call.id, prepared.args, signal, on_update
0422     )
0423     accepting_updates = False
0424     if update_jobs:
0425         await asyncio.gather(*update_jobs)
0426     return Executed(result, False)
0427 except Exception as exc:
0428     accepting_updates = False
0429     if update_jobs:
0430         await asyncio.gather(*update_jobs, return_exceptions=True)
0431     return Executed(error_result(str(exc)), True)
0432 finally:

```

```

0433         accepting_updates = False
0434
0435 async def finalize_executed_tool_call(
0436     context: AgentContext, assistant: Message, prepared: Prepared,
0437     executed: Executed, config: Config, signal: AbortSignal,
0438 ) -> Finalized:
0439     result, is_error = executed.result, executed.is_error
0440     if config.after_tool_call:
0441         try:
0442             decision = await config.after_tool_call(
0443                 assistant, prepared.call, prepared.args,
0444                 result, is_error, context, signal,
0445             )
0446             if decision:
0447                 result = ToolResult(
0448                     content=decision.content if decision.content is not None else result.content,
0449                     details=decision.details if decision.details is not None else result.details,
0450                     usage=decision.usage if decision.usage is not None else result.usage,
0451                     terminate=(
0452                         decision.terminate if decision.terminate is not None
0453                         else result.terminate
0454                     ),
0455                 )
0456             if decision.is_error is not None:
0457                 is_error = decision.is_error
0458         except Exception as exc:
0459             result, is_error = error_result(str(exc)), True
0460     return Finalized(prepared.call, result, is_error)

```

## 10. s01 / s02 / s03 一行一行映射

这里的“一行一行”不是按历史文件的绝对行号，而是按你在 s01/s02/s03 中新增的关键语句/职责逐项映射到教学 Runtime。这样即使原练习文件以后重排，映射仍然稳定。

| learn-claude-code 语句/职责           | 教学 Runtime                               | Pi 语义                                        | 演进解释                                             |
|-----------------------------------|------------------------------------------|----------------------------------------------|--------------------------------------------------|
| messages.append(user prompt)      | run_loop 入口前准备 context/new_messages      | runAgentLoop 把 prompt 合并进 currentContext     | s01 的 transcript 变成“全局 Context + 本次增量”双视图。       |
| while stop_reason == tool_use     | run_loop inner while                     | hasMoreToolCalls    pendingMessages          | 继续条件从“只有 Tool”扩展为 Tool + Human steering。         |
| response = LLM(messages, tools)   | stream_assistant_response                | transformContext -> convertToLlm -> streamFn | 模型调用被拆成 Context Engineering、协议投影和 Provider 三层。   |
| execute tool                      | execute_tool_calls                       | scheduler chooses sequential/parallel        | s01 的单工具调用升级为批量调度。                               |
| append tool_result                | context.messages.extend(result_messages) | ToolResultMessage 回到上下文                      | 反馈闭环仍然是核心不变量。                                    |
| run_bash(block.input)             | Tool.execute                             | 统一 Tool contract                             | s02 不再需要每种 Tool 写进 loop。                         |
| TOOL_HANDLERS[name]               | Tool Registry lookup                     | prepareToolCall 查 currentContext.tools       | Dispatcher 从函数映射升级成携带 schema/metadata 的 Tool 对象。 |
| safe_path(...)                    | prepare/validate_arguments               | prepareArguments + validateToolArguments     | 路径/参数处理应尽量在副作用之前。                                |
| if not check_permission: continue | before_tool_call -> Immediate            | beforeToolCall block                         | s03 的 permission 被抽象成通用 pre-hook。                |
| hard deny / rule match / approval | BeforeDecision                           | hook 返回 block/reason/terminate               | Permission 只是一类 Tool Policy。                     |
| subprocess result string          | ToolResult + ToolResultMessage           | content/details/usage/isError                | 结果从字符串升级为结构化 observation。                        |
| 无事件系统                             | emit(Event(...))                         | agent/turn/message/tool events               | Runtime 可以被 UI、持久化、Eval、Trace 订阅。                |
| 无 streaming state                 | partial -> context.messages[-1]          | message_start/update/end                     | State 与 UI 不再维护两份。                               |
| 无人工队列                             | steering/follow-up polling               | 两层 loop                                      | Human-in-loop 从 abort/restart 升级为明确时序语义。         |
| 无 post-hook                       | after_tool_call                          | finalizeExecutedToolCall                     | 结果脱敏、压缩、审计可在 Tool 外统一治理。                         |

## 11. 可以沉淀为自己 Runtime 的 know-how

### K1. Loop 只做 orchestration

LLM、Tool、Policy、Context、Provider 都通过边界注入。越少业务规则直接写进 loop，越容易测试。

### K2. Runtime State != LLM Context

先 transformContext，再 convertToLlm。保存的信息与每轮真正发送给模型的信息必须解耦。

### K3. Tool 是 Pipeline

lookup -> normalize -> validate -> pre-hook -> execute -> progress -> post-hook -> observation。

### K4. Side effect 前必须建立不变量

只有 PreparedToolCall 可以 execute；这让执行函数不再关心工具是否存在、参数是否合法、是否批准。

### K5. Error 尽量回到 observation

找不到工具、参数非法、工具异常、post-hook 异常都尽量变 ToolResult，而不是炸掉整个 Agent。

### K6. Safety 不只有 Permission

length 截断的 Tool Call 也要 fail closed；“参数能 parse”不代表“参数完整”。

### K7. 并行需要语义元数据

Tool.executionMode 是 scheduler 的输入。未来还可以增加 sideEffects/idempotent/riskLevel/resourceLock。

### K8. Event ordering 是一等公民

tool update 必须在 tool end 前 settle；result message 顺序应该稳定。否则 UI/日志/Session 会出现竞态。

### K9. Human-in-loop 需要 queue semantics

steer 与 follow-up 不是同一个动作；一个改变当前方向，一个排队到自然结束以后。

### K10. 每轮重新绑定动态配置

模型、tools、system prompt、thinking level 可能运行中变化；不要把启动时 snapshot 当成永恒真相。

### K11. Pre-hook 与 Post-hook 对称

执行前控制“能不能做”；执行后控制“结果如何进入系统”。两者构成完整治理面。

### K12. Teaching Runtime 先保持单进程

先把不变量和事件语义做对，再引入 Session Store、Sandbox、Subagent、MCP。

## 12. 练习、测试与下一步重构路线

### 12.1 建议先写的 12 个 Runtime 测试

- 模型无 tool call：只发生一次 provider 调用并 agent\_end。
- 模型 tool call 后得到 result，再发生第二次 provider 调用。
- Tool 不存在：产生 is\_error ToolResult，不抛出 run\_loop。
- 参数校验失败：handler 不被调用。
- before hook block：handler 不被调用，模型能看到 reason。
- before hook block + terminate：循环按 batch terminate 语义结束。
- stop\_reason=length 且带 tool call：任何 handler 都不得执行。
- 两个 parallel tool 的物理结束顺序反转，但 result message 顺序保持 call order。
- 任一 sequential tool 出现时整批顺序执行。
- tool update 在 tool\_execution\_end 前全部 settle。
- after hook 改写 content/is\_error 后，LLM 收到改写后的结果。
- steering 在下次 model call 前进入 context；follow-up 只在 natural stop 后进入。

### 12.2 从 460 物理行继续拆成工程模块

```
runtime/  
  messages.py      # AgentMessage / ToolResult  
  events.py        # lifecycle event model  
  loop.py          # run_loop only  
  provider.py      # stream_assistant_response boundary  
  tools.py         # registry + scheduler + pipeline  
  policy.py        # before/after hooks  
  queue.py         # steering/follow-up  
  context.py       # transform + convert  
  tests/          # invariants and race conditions
```

### 12.3 不要过早加入的东西

- Graph/Workflow DSL：先证明普通 loop 无法表达你的任务。
- 复杂 Memory Service：先把 Runtime State / Context Projection 分开。
- Multi-Agent：先建立 spawn primitive 和隔离边界。
- MCP 大工具集：先验证 tool schema 的上下文成本。
- Planner/Reflection Agent：先把 Tool observation、Context、Event 做可靠。

**核心判断：**完成本手册后，下一步最有价值的工作不是再加功能，而是给这份 Python Runtime 写测试，然后逐项加入 Session Tree、Compaction、ExtensionRunner 与 Sandbox。

### 资料基线

Pi repository: <https://github.com/earendil-works/pi>

Primary source: packages/agent/src/agent-loop.ts

Related source: packages/agent/src/agent.ts, packages/agent/src/types.ts,  
packages/coding-agent/src/core/agent-session.ts

本手册中的 Python 代码为教学重实现；函数命名与阶段划分用于帮助理解 Pi 的公开 Runtime 架构。